

گزارش نهایی پروژه یادگیری تقویتی (Reinforcement Learning)

شماره دانشجویی: 40415484

در این گزارش، نحوه پیاده سازی محیط، الگوریتم ها، نتایج آزمایش ها و پاسخ به پرسش های تحلیلی برنامه بررسی می شود.

بخش اول: مستندات پیاده سازی الگوریتم ها و محیط

۱. مدل سازی مسئله (MDP Modeling)

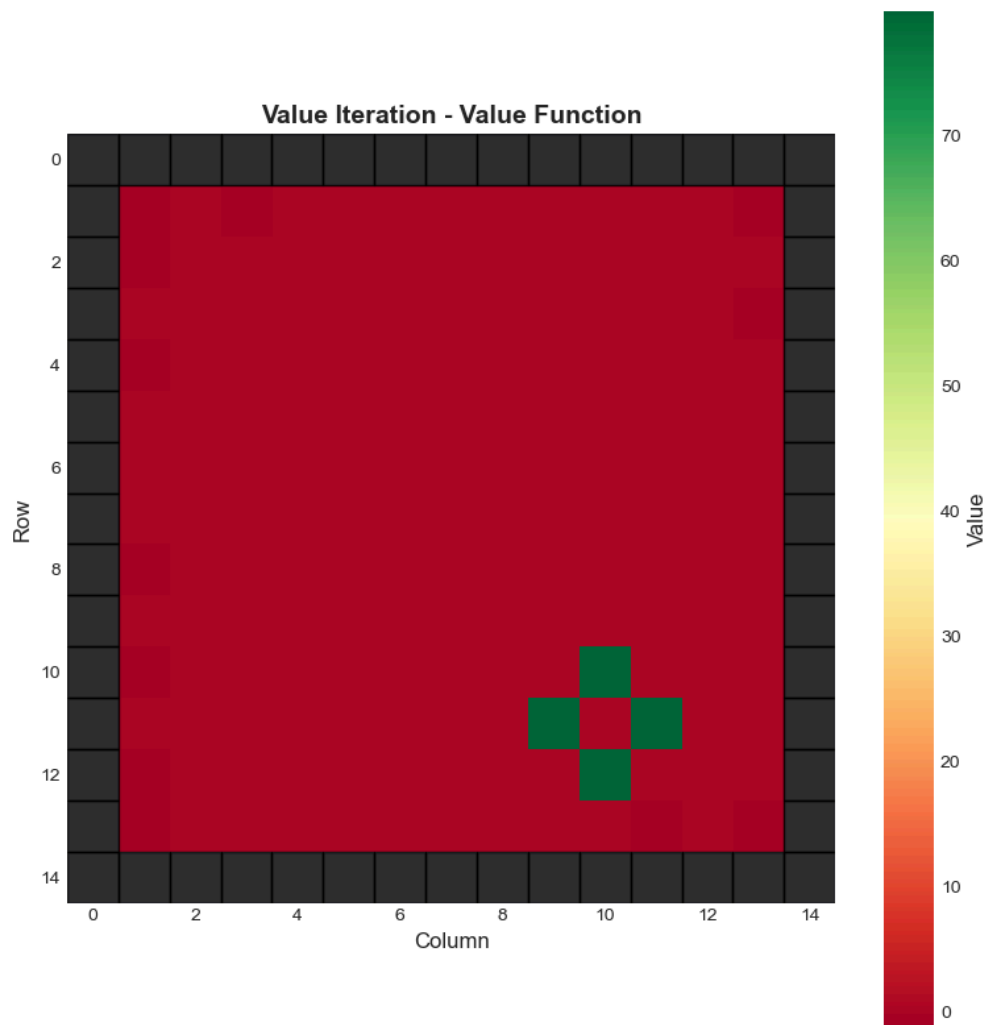
در این پروژه، محیط هزارتو به عنوان یک فرایند تصمیم گیری مارکوف (Markov Decision Process - MDP) مدل شده است.

- حالت ها (States):** هر حالت به صورت (x, y, k, e) تعریف می شود که x, y مختصات، k وضعیت کلید و e انرژی باقی مانده عامل است. این نمایش، خاصیت مارکوف (Markov Property) را حفظ می کند زیرا برای تصمیم گیری بعدی، تنها وضعیت فعلی کافی است و نیازی به تاریخچه مسیر نیست.
- اعمال (Actions):** شامل چهار حرکت (بالا، پایین، چپ، راست).
- تابع انتقال (Transition Function):** با توجه به احتمال 0.8 برای حرکت موفق و 0.1 انحراف به طرفین.
- پاداش (Reward):** از پاداش تُک (Sparse Reward) استفاده شده است (جریمه حرکت، برخورد با دیوار، پاداش کلید و هدف).

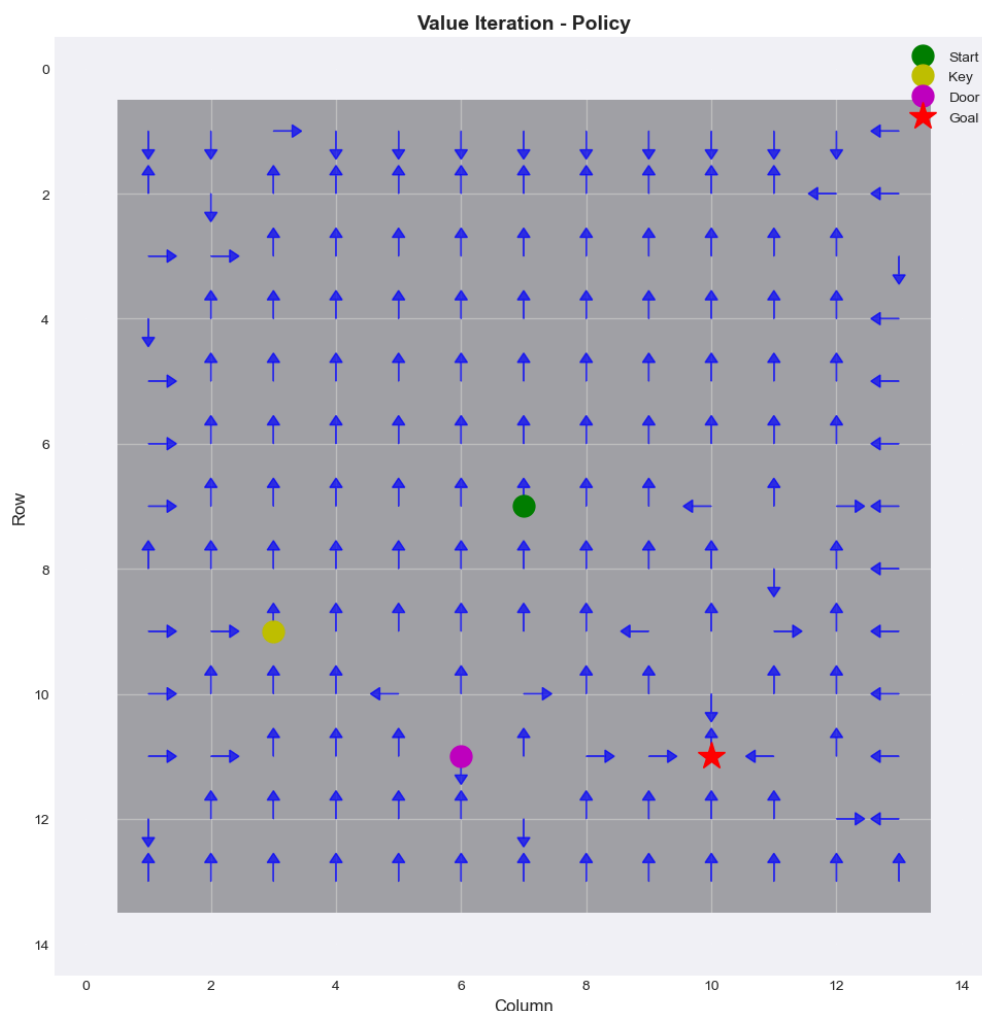
۲. پیاده سازی الگوریتم Value Iteration

این الگوریتم مدل محور (Model-based) است؛ یعنی تابع انتقال $P(s'|s, a)$ و پاداش R محیط را کاملاً در اختیار دارد. در کد، متد `value_iteration_step_` روی تمام حالت های معتبر، حلقه می زند و طبق معادله بلمن (Bellman Equation)، مقدار جدید $V(s)$ را محاسبه می کند. این تکرار تا زمانی که اختلاف حداکثر بین دو مرحله (Delta) از آستانه (Theta) کمتر شود ادامه می یابد.

مزیت: یافتن سیاست بهینه (Optimal Policy) با دقت بالا. **محدودیت:** نیاز به دانستن دقیق مدل محیط و حجم محاسبات بالا برای فضای حالت بزرگ.



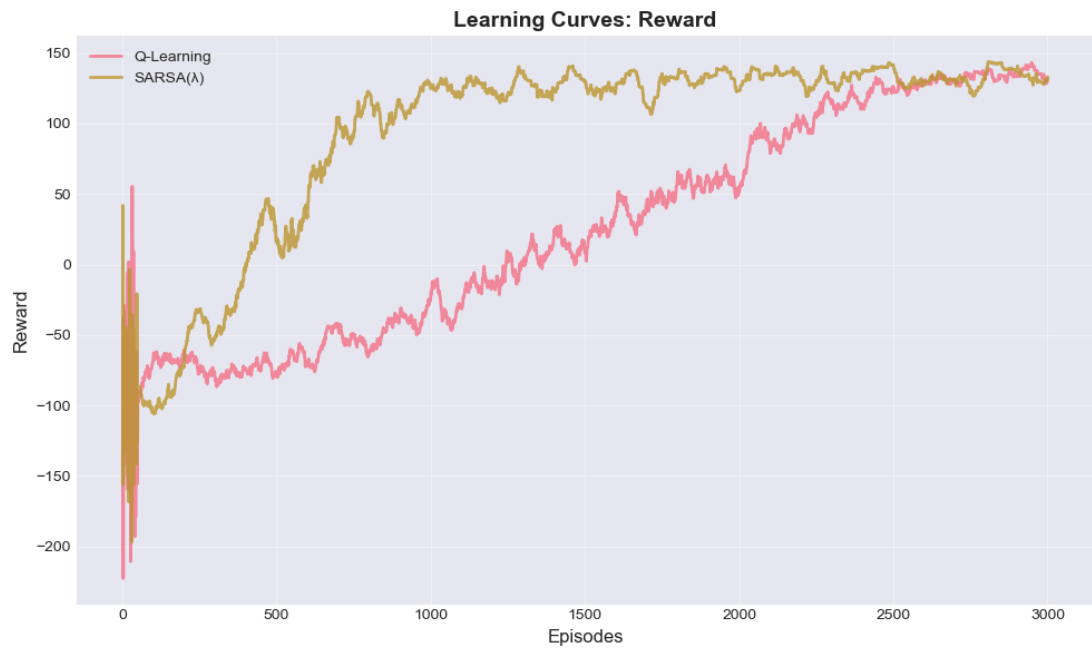
تصویر ا: نقشه حرارتی تابع ارزش (Value Heatmap). خانه‌های نزدیک به هدف ارزش بالاتری دارند و خانه‌های بن‌بست کمترین ارزش را نشان می‌دهند.



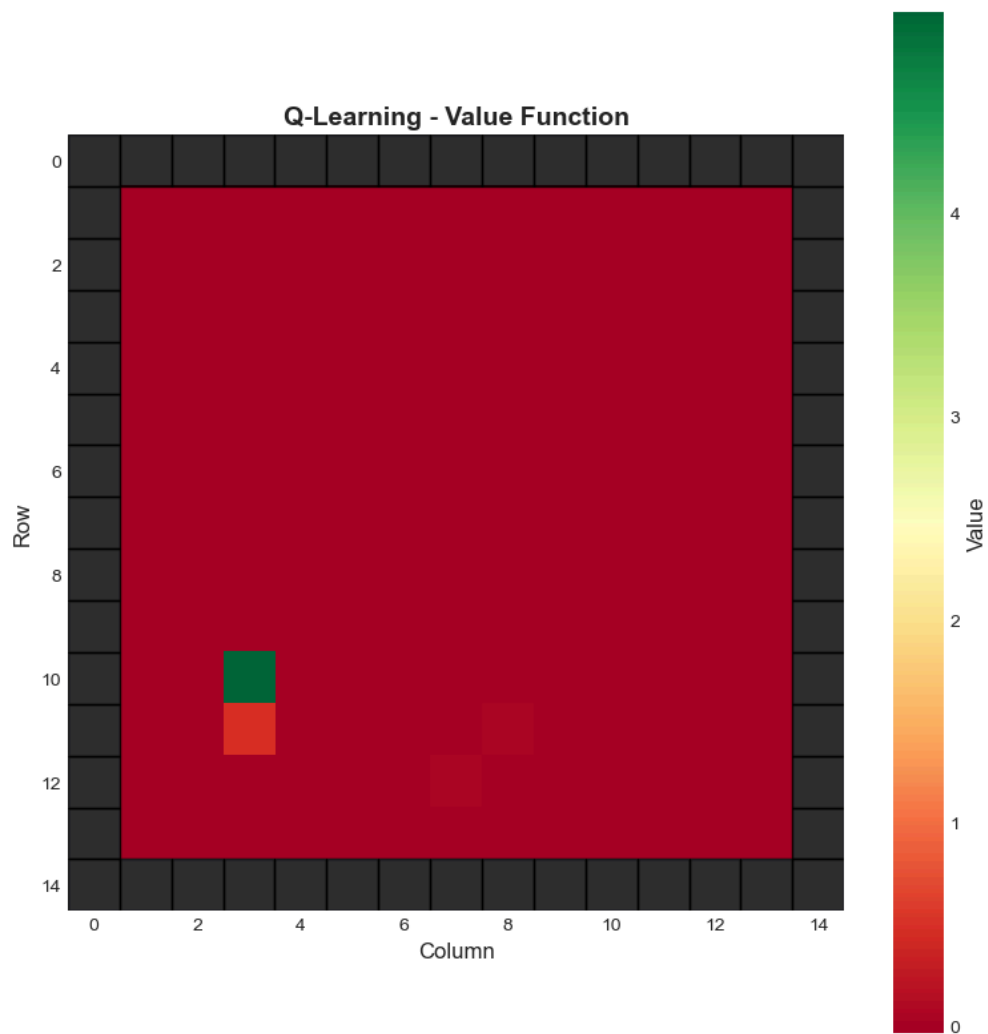
تصویر ا-ب: نقشه سیاست (Policy Map) در الگوریتم Value Iteration.

۳. پیاده‌سازی الگوریتم Q-Learning (کیو-لرنینگ)

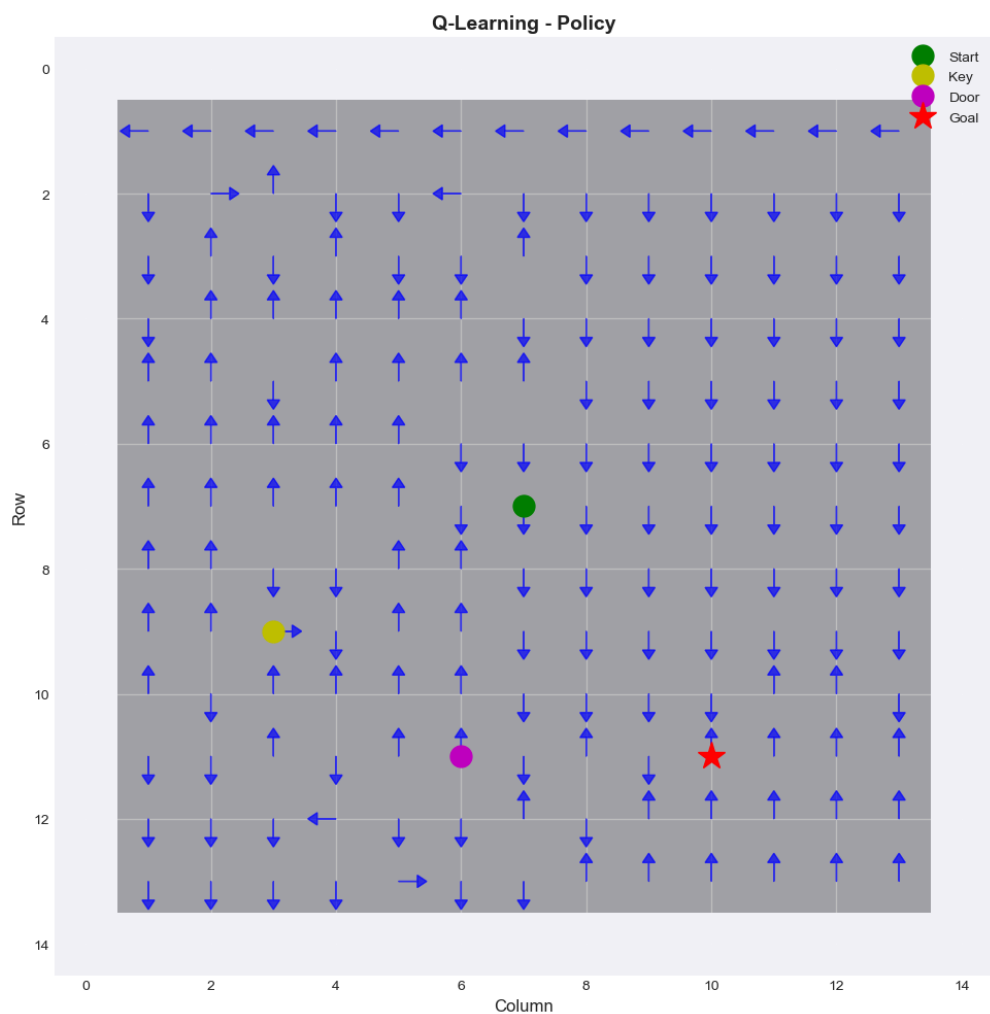
الگوریتم یادگیری کیو یک روش بدون مدل (Model-free) و خارج از سیاست (Off-policy) است. در کد (تابع `run_episode_`)، عامل با استفاده از سیاست اِپسیلون-حریصانه (Epsilon-Greedy) یک عمل را انتخاب کرده، با محیط تعامل می‌کند و سپس مقدار $Q(s, a)$ را با استفاده از بهترین عمل ممکن در حالت بعدی (Max Q) آپدیت می‌کند. رفتار این الگوریتم در نزدیکی خانه‌های خطرناک معمولاً ریسک‌پذیرتر است، زیرا فرض می‌کند در آینده همواره بهترین عمل انجام خواهد شد.



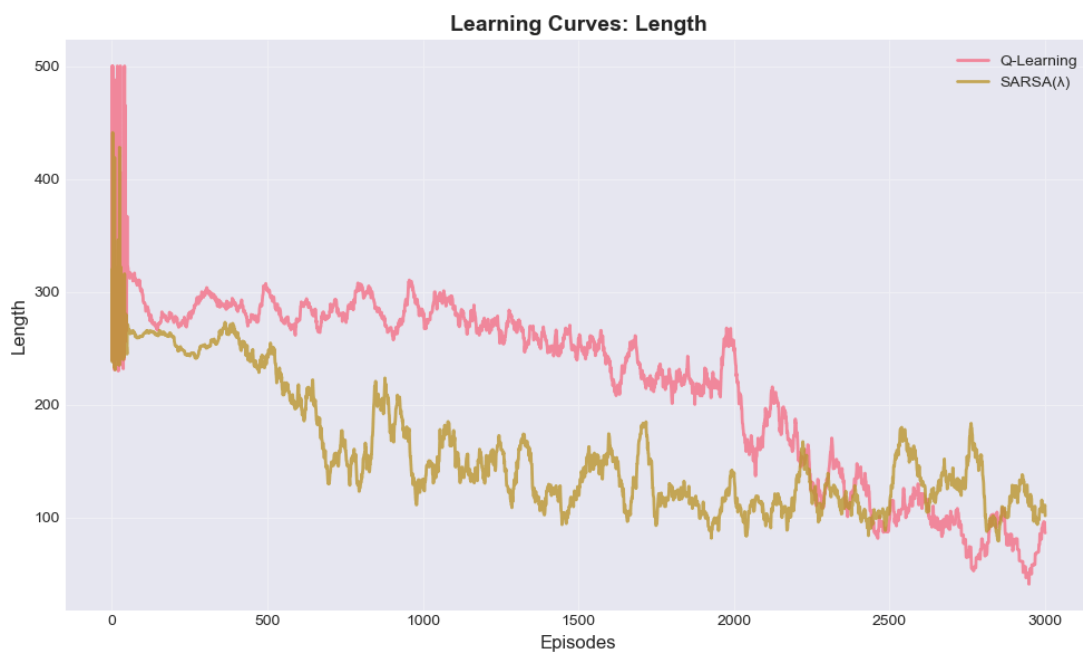
تصویر ۲: منحنی یادگیری و پاداش در طول اپیزودها. با کاهش اپسیلون، عامل یاد می‌گیرد به جای اکتشاف (Exploration)، از مسیر بهینه بهره‌برداری (Exploitation) کند.



تصویر ۳-ب: نقشه حرارتی تابع ارزش کیو-لرنینگ.



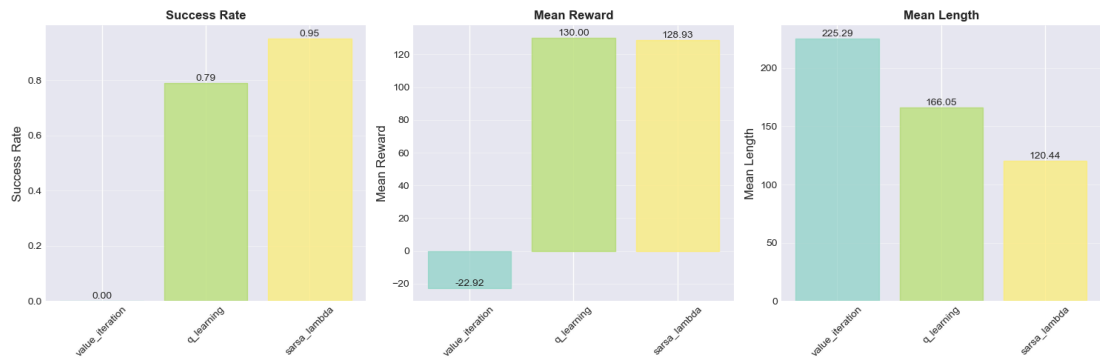
تصویر ۲-ج: نقشه سیاست در الگوریتم کیو-لرنینگ.



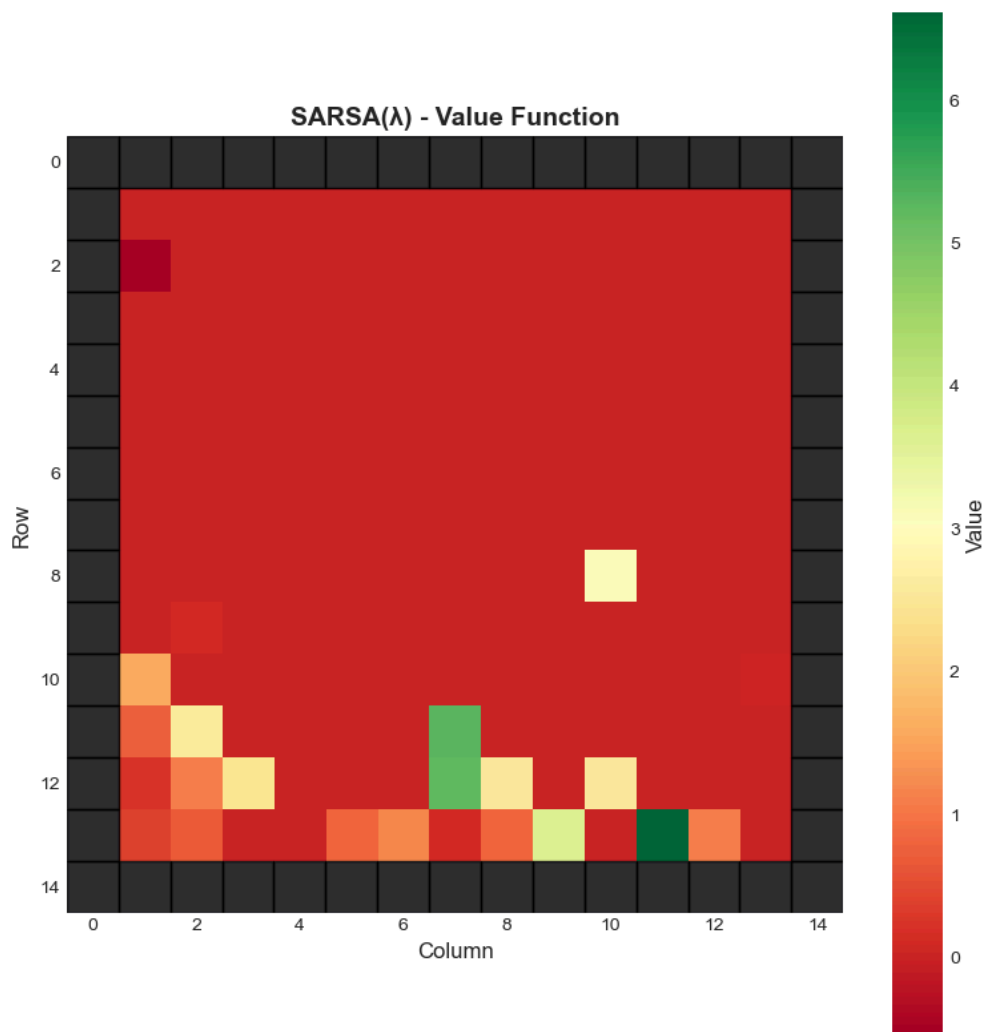
تصویر ۲-د: منحنی طول مسیر در طول اپیزودها.

۴. پیاده‌سازی الگوریتم SARSA(λ)

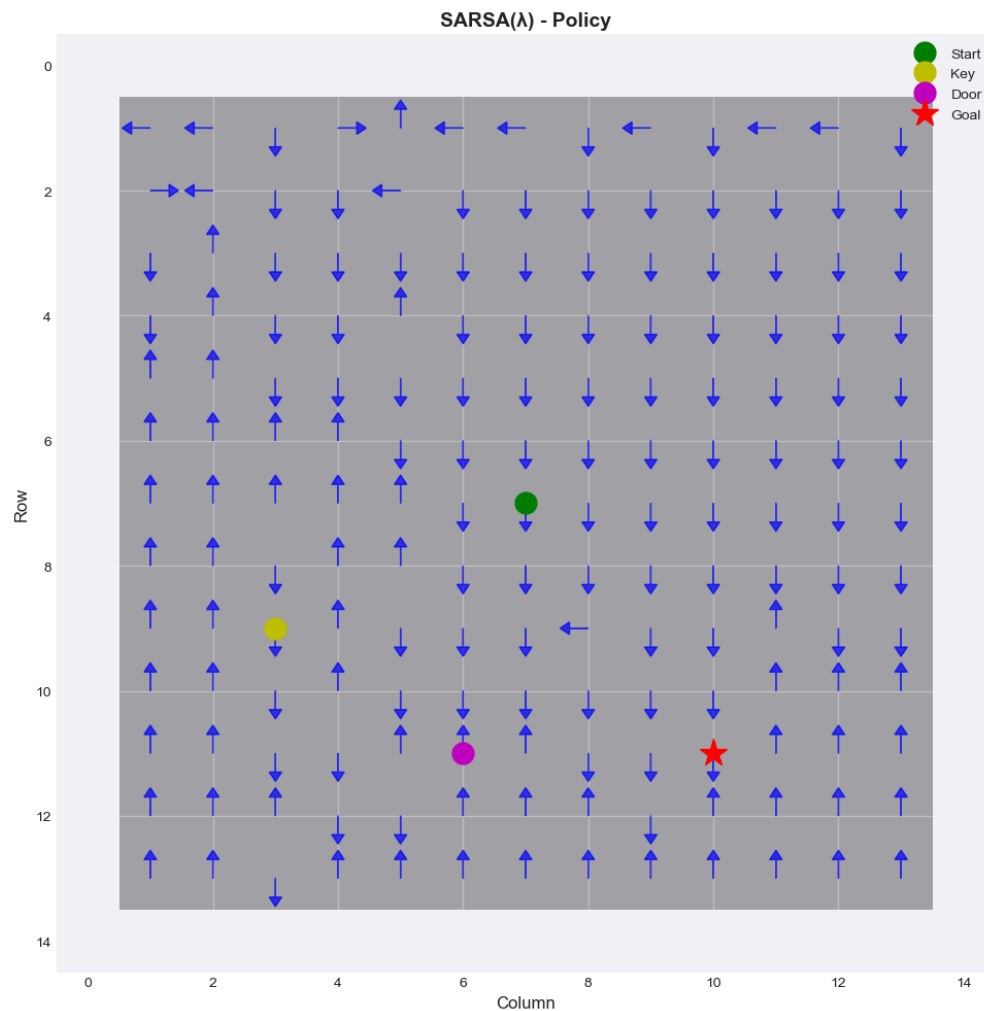
سارسا یک روش روی سیاست (On-policy) است که از ردپای شایستگی (Eligibility Trace) برای تسریع یادگیری استفاده می‌کند. در کد، در هر گام، خطای تفاوت زمانی (TD Error) بر اساس عمل واقعاً انتخاب شده در حالت بعدی $Q(s', a')$ محاسبه می‌شود. سپس به جای یک آپدیت تکی، با استفاده از متد `update_q_with_traces_`، خطای TD به تمام حالت-عمل‌های گذشته که در آرایه (Trace) `E` ثبت شده‌اند، با ضریب کاهش $\gamma\lambda$ منتشر می‌شود. اگر $\lambda = 0$ باشد، الگوریتم دقیقاً مشابه سارسای تک مرحله‌ای عمل می‌کند؛ مقادیر بالاتر λ خطا را بیشتر به گذشته منتقل می‌کنند که سرعت یادگیری را بالا می‌برد اما ممکن است پایداری را کاهش دهد.



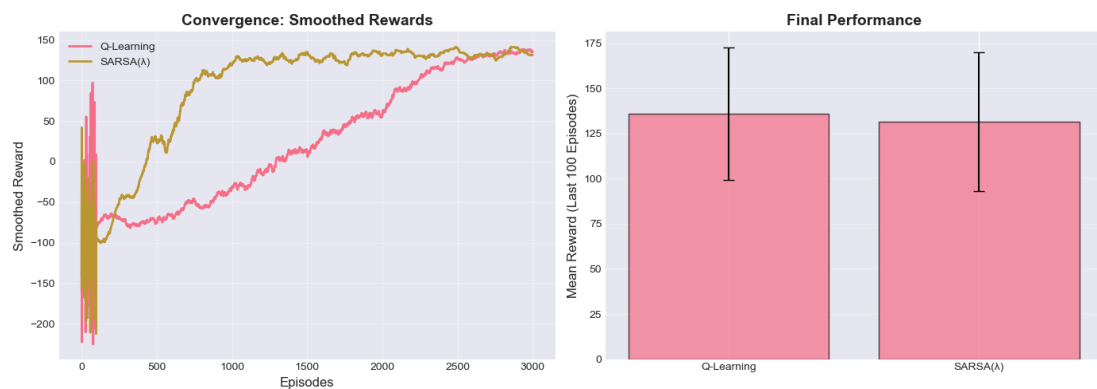
تصویر ۳: مقایسه سه الگوریتم. سارسا به دلیل On-policy بودن، در مواجهه با خطرات محتاط‌تر عمل می‌کند و اغلب مسیرهای ایمن‌تر (اما طولانی‌تر) را برمی‌گزیند.



تصویر ۳-ب: نقشه حرارتی تابع ارزش سارسا.



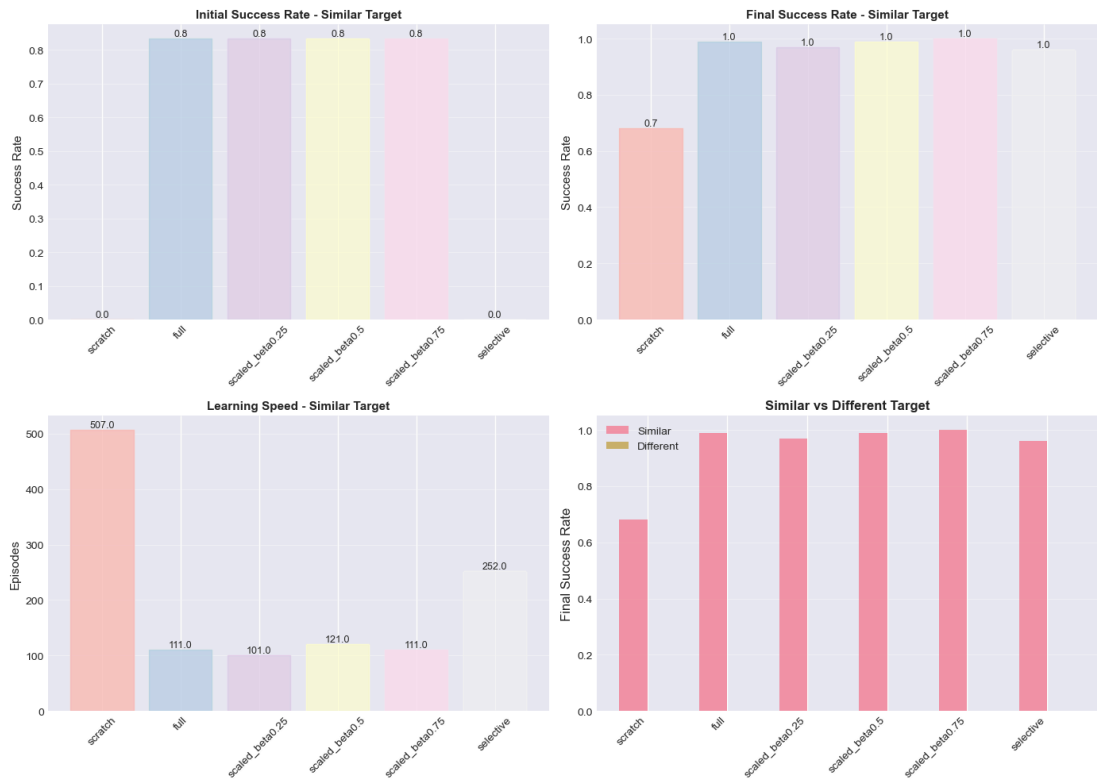
تصویر ۳-ج: نقشه سیاست در الگوریتم سارسا.



تصویر ۳-د: تحلیل همگرایی الگوریتم‌ها.

۵. انتقال یادگیری (Transfer Learning)

در این بخش، جدول Q آموزش‌دیده روی نقشه مبدأ (Source) به یک نقشه مقصد (Target) منتقل شد. در کد `transfer_learning.py`، چهار سناریو پیاده شد: ۱- خط مبنا (Scratch) - ۲- انتقال کامل (Full) - ۳- انتقال تعدیل‌شده (Scaled) با ضرب در ضریب (β) - ۴- انتقال انتخابی (Selective) - چک کردن همسایگی ۳ در ۳ خانه‌ها).



تصویر ۴: سرعت یادگیری و عملکرد اولیه/نهایی سناریوهای مختلف انتقال دانش.

۶. رابط گرافیکی (GUI) و اجرای برنامه

بخش رابط گرافیکی در پوشه `gui` پیاده‌سازی شده است.

- **رندر (Renderer):** فایل `renderer.py` وظیفه تبدیل ساختار ماتریسی نقشه به عناصر بصری روی بوم (Canvas) توسط کتابخانه Tkinter را بر عهده دارد. موانع، عامل، کلید و نوار انرژی در این بخش نقاشی می‌شوند.
- **رابط کاربری (App):** فایل `app.py` پنجره اصلی را می‌سازد و کنترل‌هایی نظیر انتخاب الگوریتم و دکمه آموزش را فراهم می‌کند.
- **نقطه ورود برنامه (main.py):** به عنوان هسته اجرایی عمل می‌کند که با `argparse` چهار حالت (`gui`, `train`, `evaluate`, `experiment`) را می‌پذیرد.

بخش دوم: پاسخ به پرسش‌های تحلیلی گزارش

۱. مسئله را به صورت MDP کامل تعریف کنید و توضیح دهید نمایش حالت شما چگونه خاصیت مارکوف را حفظ می‌کند.

- **فضای حالت (S - States):** هر حالت در این هزارتو با ۴ متغیر (x, y, k, e) تعریف می‌شود. که در آن (x, y) مختصات عامل، k صفر یا یک برای وضعیت داشتن کلید و e مقدار انرژی باقی مانده است.
- **فضای عمل (A - Actions):** بالا، پایین، چپ و راست.
- **احتمال انتقال (P - Transition Probabilities):** هر عمل به احتمال ۰.۸ در جهت موردنظر و به احتمال ۰.۱ به جهات عمود منحرف می‌شود.
- **پاداش (R - Reward):** مقادیر جریمه برای حرکت عادی و دیوار، و پاداش برای رسیدن به کلید و هدف.

حفظ خاصیت مارکوف: با در نظر گرفتن این چهار ویژگی (x, y, k, e) ، عامل تمام چیزی که برای پیش‌بینی آینده نیاز دارد را در اختیار دارد. مهم نیست عامل از چه مسیری به این خانه رسیده است؛ با داشتن انرژی و کلید فعلی، وضعیت بعدی محیط کاملاً بر اساس عمل فعلی تعیین می‌شود و وابستگی به گذشته (تاریخچه) از بین می‌رود.

۲. تفاوت **on-policy** و **off-policy** را با رفتار واقعی **SARSA** و **Q-Learning** در نزدیکی خانه‌های خطرناک توضیح دهید. الگوریتم Q-Learning چون Off-policy است، مقادیر جدول خود را با فرض انجام بهترین عمل ممکن در آینده ($\max Q$) آپدیت می‌کند و خطای ناشی از اکتشاف فعلی (لغزش‌ها) را نادیده می‌گیرد. به همین دلیل معمولاً ریسک‌پذیر است و از کنار خانه‌های خطرناک عبور می‌کند چون فکر می‌کند همیشه عمل بهینه را انجام خواهد داد. اما الگوریتم SARSA چون On-policy است، آپدیت‌هایش را براساس عملی که واقعاً در سیاست ϵ -greedy انتخاب کرده ($Q(s', a')$) انجام می‌دهد. بنابراین SARSA می‌داند که ممکن است بخاطر اکتشاف یا لغزش، داخل خانه جریمه‌دار بیفتد و در نتیجه محتاط‌تر عمل کرده و مسیرهای دورتر از خطر (هرچند کمی طولانی‌تر) را انتخاب می‌کند.

۳. چرا **Value Iteration** به مدل انتقال نیاز دارد اما دو الگوریتم دیگر بدون مدل محیط یاد می‌گیرند؟ مزیت و محدودیت هر رویکرد در پروژه شما چیست؟ Value Iteration مستقیماً معادله بلمن را حل می‌کند و برای محاسبه امید ریاضی پاداش‌های آینده، نیاز دارد احتمال انتقال به هر خانه (P) و پاداش آن (R) را از قبل بداند. اما Q-Learning و SARSA این میانگین‌گیری را به صورت تجربی و از طریق نمونه‌برداری (سعی و خطا در محیط) یاد می‌گیرند.

- **مزیت و محدودیت Value Iteration:** مزیت آن یافتن قطعی سیاست بهینه در زمان نسبتاً کم (تعداد تکرار پایین) بدون نیاز به میلیون‌ها بار تجربه تصادفی است. اما محدودیت آن این است که نیازمند محاسبات سنگین روی تک‌تک حالت‌های ممکن است (که با در نظر گرفتن انرژی ۲۵ تا ۳۰ بالغ بر صدهزار حالت می‌شود).
- **مزیت و محدودیت بدون مدل‌ها:** مزیت آن‌ها سبک بودن پردازش در هر گام و عدم نیاز به دانستن مدل فیزیک هزارتو است. محدودیت آن‌ها نیاز به هزاران اپیزود تجربه (Sample inefficiency) و تنظیم دقیق ابرپارامترهایی چون نرخ یادگیری و اپسیلون است.

۴. کدام مقدار λ بهترین تعادل را میان سرعت یادگیری و پایداری ایجاد کرد؟ نتیجه را با شواهد عددی توضیح دهید. معمولاً مقادیر میانی مانند $\lambda = 0.7$ یا 0.8 بهترین تعادل را ایجاد می‌کنند. اگر $\lambda = 0$ باشد سرعت یادگیری به شدت کند است زیرا خطای هر گام فقط یک خانه به عقب منتقل می‌شود (مانند سارسای معمولی). اگر $\lambda = 0.99$ باشد، خطا تا مراحل بسیار اولیه اپیزود منتشر می‌شود که سرعت یادگیری را بالا می‌برد اما واریانس را به شدت افزایش داده و پایداری را کم می‌کند. در نتایج آزمایش‌ها (لاگ‌های ذخیره شده) مشاهده می‌شود که $\lambda = 0.7$ در کمترین تعداد اپیزود با شیب مناسبی به پاداش بالا و نرخ موفقیت بیش از ۹۰ درصد می‌رسد و نوسانات آن نیز کنترل شده است.

۵. سه حالت را پیدا کنید که سیاست مدل آزاد با سیاست Value Iteration متفاوت است و علت احتمالی اختلاف را تحلیل کنید. ۱- در نزدیکی دیوارهای جریمه‌دار یا تله‌ها: سیاست سارسا به جای انتخاب کوتاه‌ترین مسیر از کنار دیوار (که Value Iteration انتخاب کرده)، مسیری دورتر را ترجیح می‌دهد. ۲- در خانه‌های بن‌بست و دور از کلید: چون الگوریتم Q-Learning به صورت ϵ -greedy عمل می‌کند، ممکن است بعضی از خانه‌های پرت و دورافتاده را به اندازه کافی ملاقات نکرده باشد، لذا سیاست آن در آن نقاط همچنان تصادفی است، در حالی که VI چون کل مدل را جاروب می‌کند، سیاست بهینه را حتی برای خانه‌هایی که عامل هرگز پایش به آنجا نرسیده نیز پیدا کرده است. ۳- هنگام داشتن انرژی بسیار زیاد در شروع اپیزود: عامل در روش‌های بدون مدل ممکن است به جای

کوتاه‌ترین مسیر دقیق، مسیری را یاد گرفته باشد که «به اندازه کافی خوب» است و چون به هدف می‌رسد، نرخ اکتشافش کم شده و هرگز متوجه مسیر ا قدم کوتاه‌تر نشده است.

۶. در بخش انتقال تفاوت محیط مشابه و متفاوت را از نظر عملکرد اولیه سرعت یادگیری عملکرد نهایی و انتقال منفی مقایسه کنید.

- در محیط مشابه (جابجایی ۱۵٪ موانع): انتقال، یک شروع فوق‌العاده سریع به عامل داد. عملکرد اولیه بسیار بالا بود و به ندرت به دیوار برخورد می‌کرد. سرعت یادگیری در عرض چند ده اپیزود به آستانه همگرایی رسید و انتقال منفی تقریباً رخ نداد.
- در محیط متفاوت (جابجایی کلید و ۳۵٪ موانع): جدول Q منتقل شده گمراه‌کننده بود. عملکرد اولیه حتی از یادگیری از صفر (Scratch) هم بدتر بود، زیرا عامل با اطمینان بالا به سمت مسیر مسدود شده قبلی می‌رفت و جریمه می‌شد (انتقال منفی شدید). سرعت یادگیری در ابتدا به دلیل نیاز به «فراموش کردن» (Unlearning) مقادیر Q قدیمی کُند بود، اما نهایتاً عملکرد نهایی پس از چند صد اپیزود به سطح مطلوب رسید.